

Big Data Based IMDb Movie Review Engine

Shaher Bano Sani, Aimen Atif
Department of Computer Engineering, UET Taxila

ABSTRACT

The rise of user-generated material on social media sites such as YouTube, Netflix, and Twitter have made it difficult to organize and handle massive amounts of user-generated film reviews. The days of manually going through all the data are over. In order to address this problem, we concentrate on utilizing big data technologies particularly from Apache Pig environment, which make it possible to store, process, and analyze enormous amounts of data effectively while maintaining scalability and real-time performance. In this paper, we present a systematic method for creating a review engine and movie recommendation system combining big data, machine learning, and sentiment analysis. The method distinguishes between good and negative user input by using sentiment analysis to arrange movie reviews into distinct files based on their sentiments. This module extracts sentiments from textual reviews and categorizes them as favorable or negative using cutting-edge natural language processing algorithms. Cosine similarity technique is incorporated into the system to improve the recommendation capabilities. This makes it possible to analyze user preferences and movie features, which leads to the alignment of related films based on cast and theme. By giving consumers, a platform to voice their ideas and provide precise and individualized recommendations, the proposed system seeks to improve the movie-watching experience.

Keywords: Big Data technologies, Apache Pig, Movie Recommendation Systems, user-generated content, Review Engine, sentiment analysis, Cosine Similarity

1. INTRODUCTION

User-generated reviews and movie material have grown exponentially in the ever-expanding world of cinema. This wealth of data, however, makes it difficult for moviegoers to find pertinent films and make wise choices based on user reviews. By offering individualized recommendations and insights into user opinion, review engines and movie recommendation systems are essential in tackling these issues.

This research effort aims to further improve movie recommendation algorithms by utilizing big data technology. Scalability and real-time performance can be ensured by efficiently storing, processing, and analyzing vast amounts of movie-related data by utilizing big data. This project's main goal is to create a review engine and movie recommendation system that uses cutting-edge methods to address the issues that moviegoers encounter. The system can automatically estimate the feelings from textual

reviews and categorize them as positive or negative by using sentiment analysis. The sentiment analysis module employs sophisticated natural language processing methods to ascertain the implicit feelings and viewpoints conveyed by users. The system analyzes movie attributes and user preferences using a CNN architecture in addition to sentiment analysis. This study makes it possible to compare films that are comparable in terms of cast, topic, and other pertinent elements. Through the integration of various methodologies, the system can produce precise and customized movie suggestions, improving users' movie discovery experience.

This project makes use of big data technologies like Apache Pig to process and analyze massive amounts of movie reviews efficiently. These tools are excellent at managing the intricacies and variances of big and varied datasets, allowing thorough textual data analysis as well as effective

storage and retrieval processes. Additionally, the project seeks to provide quick and scalable access to movie data by optimizing data storage, indexing, and querying processes. To determine areas for optimization and improvement, a benchmarking process will be conducted against current practices.

2. LITERATURE REVIEW

The term Big Data refers to the massive amount of both structured and unstructured data produced by a variety of sources, including social media and internet platforms, electronic devices. It is one of the major applications of parallel and distributed computing. Capturing, storing, analyzing, sharing, transferring, visualizing, querying, updating, information privacy, and data source are some of the concerns associated with big data analysis.

One of the major distinctive characteristics of big data includes its “4Vs” generally referred to as volume, variety, velocity, and veracity of data.

Apache Hadoop, a free to use and open-source platform enables the users to process, analyze and store large amounts of data in a distributed way. The Hadoop framework itself is mostly written in Java, while shell scripts and some native C code are utilized for command line utilities. It makes use of commodity hardware integrated with each other. HDFS is a fault-tolerant file system for the processing of enormous files. Developed for the Hadoop architecture, it is a distributed, scalable, and portable file system with a Java foundation. It works by partitioning huge files into blocks and then distributing them over the cluster.

Apache Pig is a high-level data flow language and open-source processing model that is used for the analysis of larger datasets in an easy and faster way. Pig has the ability to run its Hadoop tasks in Apache Spark, Apache Tez, or MapReduce. It enables effortless data summarization, reporting, querying and analysis of large amounts of data. It provides abstraction over MapReduce and provides an engine for executing dataflows parallelly. Pig Latin is the scripting language of Pig that allows the users to perform data transformations and computations in a clear and understandable way. Its purpose is to

make handling big datasets on Hadoop easier and more automated. Pig Latin scripts take use of Hadoop's scalability and fault-tolerance by running concurrently throughout a distributed cluster.

A recommendation system is an information filtering system designed to anticipate and propose relevant items or content to users. Its main objective is to offer personalized suggestions that align with the preferences and interests of users that are based on past similarities with other users as well.

Content based recommendation systems are used to analyze the similarity between different users or items to provide recommendations. It analyzes metadata to identify similar movies based on attributes such as keywords, taglines, and director. These systems examine the attributes and descriptions of items to identify common characteristics among a user's favorite items, which are then stored in the user's profile. In a specific case, it utilizes cosine similarity as a prediction method. It considers various attributes such as keywords, taglines, director, and cast from the sources.

Collaborative filtering is a technique used in recommendation systems to provide personalized recommendations to users. It relies on analyzing the behavior and preferences of a group of users to make recommendations to other users. The underlying idea is that users who have similar preferences or exhibit similar patterns of behavior are likely to have similar tastes and preferences for items.

The hybrid recommender system combined the concepts of both content-based and collaborative recommendation systems to generate predictions. The output of the hybrid recommender system is a list of items, sorted based on the expected ratings by the specific user.

Sentiment analysis, also referred to as opinion mining, is a computational approach employed to ascertain the sentiment or emotional tone conveyed within text-based data. It entails the examination of textual content to identify and extract subjective information, including positive, negative, or neutral sentiments, as well as specific emotions like

happiness, anger, or sadness. This valuable information enables us to customize our recommendations to align with their individual tastes and requirements more accurately.

For sentiment classification, machine learning algorithms such as Logistic Regression, Random Forest, and Support Vector Machines (SVM) can be utilized. The process can involve multiple steps, including data collection, cleaning, and preprocessing, feature extraction, sentiment classification, and evaluation, utilizing techniques like tokenization, stemming or lemmatization, part-of-speech tagging, and sentiment lexicons, conducting of n-gram analysis to explore word sequences, and transforming the text data into numerical vectors using vectorization techniques.

RELATED WORK

Verma, [3] proposed a recommendation system using Hadoop architecture to analyze and process large amounts of data from social media platforms in the form reviews, opinions, and remarks. It defined how different types of textual data could be handled to extract meaningful information from it and also presented a hybrid filtering approach for the system. They had used Apache Hive for the processing of large movie ratings dataset and designed its Mahout Interfaces to analyze the performance with different file sizes in a distributed computing environment using the Hadoop framework.

Hongyan Liu [4] presented a recommendation algorithm that determined the user's preferences by comparing ratings and opinions. This approach took into account both explicit and implicit assessments, which helped with the sparseness of the data issue. They focused on the hybrid filtering technique in the paper, to filter different types of reviews and opinions.

Sarker, Kamal Uddin, et al. [5], focused on the role of data in AI, machine learning, and data science applications, particularly through the use of web data extraction and Web Scraping techniques. They discussed the implications of biased user ratings and conducted an analysis of movie ratings from IMDB and Amazon. They emphasized the

importance of data quality and quantity in influencing machine learning algorithms. The paper presented a novel clustering model based on two different rating systems of IMDB movie data and found that only 42.4% of the extracted dataset records were accepted for research purposes after cleaning.

Pang et al. [6] classified the emotional polarity of the movie commentary text into positive and negative categories using the maximum entropy (ME) and part of speech (POS) N-gram grammar (n-gram). This was done using supervised learning techniques in machine learning. Turney used machine learning's unsupervised learning technique to investigate the polarity of text emotions [7]. The movie reviews dataset yielded an accuracy of 65.83% for the approach.

Wang, Y. [10] proposed a Sentiment-Enhanced Hybrid Recommender System. They used Euclidean distance to measure the similarity between the items.

3. MATHEMATICAL MODEL

A mathematical model is provided in this study that combines an IMDb movie review engine with a sentiment analysis-based content-based recommendation system based on cosine similarity. It utilizes both of these techniques, to improve the movie selection process and offer more precise and individualized suggestions. The suggested approach combines the best elements of sentiment analysis—which evaluates the tone of movie reviews—with content-based filtering, which compares films according to their characteristics.

After the pre-processing of data obtained to be used for the recommendation system, it is necessary for it to be converted into vector form. For this purpose, count vectorizers or TF-IDF vectorizers are used. The TF-IDF vectorization process converts the textual data (combine features) into a matrix representation, where each row represents a movie, and each column represents a term (word) in the entire corpus [23].

Every word in a document is represented by the inverse document frequency, which is the product

of the term frequency and the logarithm of the division of the total number of documents in the corpus by the total number of documents in which the word I appears in the corpus. When viewing review texts as documents, this entails multiplying the number of times a word occurs in a review text by the logarithm of dividing the total number of reviews in the corpus by the number of reviews in which the term appears. The formula for TFIDF is expressed mathematically in equation 1.1.

$$C(t, d) \times \log \frac{N_d}{N_{dt}} \quad (1.1)$$

Where (t, d) represents the raw count of a word/term t in a document d , N_d represents the total number of documents in the corpus, and N_{dt} represents the number of documents containing the term t .

The content-based movie recommendation system uses the cosine similarity technique to measure the similarity between two vectors and calculating the cosine of the angle between them.

In the case of Movie Recommendation Systems, the feature vector represents the attributes or content of the movies. The cosine similarity value ranges from 0 to 1, where 0 indicates no similarity, and 1 represents perfect similarity [24]. Equation 1.2 shows the Cosine Similarity between two items x and y is given as:

$$\frac{\sum x_i y_i}{\sqrt{\sum x_i^2} \sqrt{\sum y_i^2}} \quad (1.2)$$

If we denote the closest match as $M_closest$ so the index of the closest match movie in the dataset is obtained using the equation 1.3:

$$\begin{aligned} &index\ of\ movie = \\ &movie_data[movie_data_title == \\ &M_closest][\text{'index'}].values[0] \end{aligned} \quad (1.3)$$

The similarity scores between the selected movie ($M_closest$) and all other movies are retrieved from the cosine similarity matrix (S_cosine). The equation used is 1.4:

$$similarity_score = list[enumerate(S_{cos[inde\ of\ movie]})] \quad (1.4)$$

The similarity scores are sorted in descending order to obtain a list of similar movies. The equation 1.5 is used for sorting:

$$\begin{aligned} &sorted_similar_movies = \\ &sorted(similarity_score, key = \\ &\lambda x: x[1], reverse = True) \end{aligned} \quad (1.5)$$

The recommender gives a prediction based on the weighted rating (WR) calculated. It takes in vote count and vote average from Movies metadata.

$$Weighted\ Rating\ (WR) = (v/v + m.R) + (m/v + m.C) \quad (1.6)$$

Finally, the code selects the top-ranked similar movies based on their similarity scores and presents them as recommendations to the user. For the movie review system, the technique of logistic regression is used for the purpose of sentiment analysis. The logistic regression function is defined as equation 1.7.

$$h(x, \theta) = \frac{1}{(1 + \exp(-\theta^T x))} \quad (1.7)$$

where θ is the parameter vector and x is a feature vector. The model learns the parameter vector θ by minimizing the logistic loss function given in equation 1.8:

$$J(\theta) = \frac{-1}{n \sum [y(i) \times \log(h(x(i); \theta)) + (1 - y(i)) \times \log(1 - h(x(i); \theta))]} \quad (1.8)$$

where $x(i)$ and $y(i)$ are the i -th feature vector and sentiment label, respectively.

For the sentiment prediction of reviews entered by the user, let us denote the sentiment of a text as follows:

Positive sentiment: $S = 1$

Negative sentiment: $S = -1$

Neutral sentiment: $S = 0$

The equations representing the conditions for sentiment classification based on sentiment scores and predefined threshold values:

1. If sentiment score $> T_{pos}$, then classify as positive sentiment
2. If sentiment score $< T_{neg}$, then classify as negative sentiment
3. Otherwise, if $S = 0$, then classify as neutral sentiment

The sentiment label is denoted by S in these equations, the calculated sentiment score for the provided text is represented by `sentiment_score`, the threshold values for positive and negative sentiment are T_{pos} and T_{neg} , respectively.

A sentiment is categorized as positive if its score is higher than the positive threshold. And is categorized as negative if the sentiment score is less than the negative threshold. On the other hand, if neither condition is met, then classify the review as neutral.

4. PROPOSED SCHEME

4.1 Dataset

We have collected the data from Internet Movie Database (IMDB), which is an online database of information related to movies, pictures, video games etc. We have retrieved data on movies, including title, genre, release year, ratings, reviews, and user demographics. The link to the dataset is: <https://github.com/SK7here/Movie-Review-Sentiment-Analysis/blob/master/IMDB-Dataset.csv>
https://www.kaggle.com/code/rounakbanik/movie-recommender-systems/input?select=movies_metadata.csv

4.2 Proposed System

The proposed system takes features from content-based recommendation to come up with a system which is more personalized for users. Basically, there are two major algorithms working in the whole system. Firstly, the content-based recommendation system for movies is used to provide recommendations to the users based on the movies they previously used to watch, and the sentiment analysis algorithm is employed to predict

the sentiments expressed in user reviews for each movie.

To facilitate seamless user interaction, both the content-based recommendation and sentiment analysis algorithms are integrated into a user interface. This interface empowers users to not only watch movies and receive personalized recommendations based on their preferences but also provides the functionality to predict their own reviews using sentiment analysis.

A. Data Analysis through Apache Pig:

The movie metadata could be taken in any structured or unstructured form. The schema will be created in order to determine the relevant columns, which will include movie attributes such as original title, genres, and index.

The results of the analysis will be stored separately for each genre. Each genre's top 10 films are stored in distinct files on the HDFS, allowing easy access and further examination. Users can consult these files to view the highest-rated films for different genres, along with their respective genres and average vote numbers.

The data preparation stage involves the cleaning, and tokenizing of data and assigning of sentiment score using a sentiment dictionary. Average sentiment ratings are calculated, and positive and negative reviews are filtered based on their average sentiment ratings, generating two separate datasets. The results of the analysis, including the review ID, review text, and average sentiment rating, for both positive and negative reviews, are stored in different files using the STORE command for further analysis.

B. Content-based movie recommendation system:

Firstly, the movie metadata is inspected using various Python commands to get insights from it. The attributes, values, and data types of movies, along with their descriptive statistics such as count, mean, standard deviation, are calculated. Other than

this null values in each attribute, are identified to help in preprocessing.

As the algorithm only focuses on genres and overviews of movies, only these are extracted out of the others by means of feature selection step.

To simplify, we only use the attributes of ID, title, genres, and overviews. Next, we perform the text vectorization step, to convert the textual data (combination of genre and overview attributes) into a numerical representation. The vectorizer is fitted to the text data, analyzing it and building a vocabulary. The resulting sparse matrix representation is converted into a dense array format using the `.toarray()` method.

Cosine similarity technique is then applied to calculate the similarity between different movies based on their combined genres and overviews. Similarity values are calculated using the feature vectors obtained from text vectorization.

The resulting array represents the similarity scores between movies and is utilized for recommending similar movies.

Finally, in the recommendation step, the similarity values from the previous step are used to recommend movies to users. The function finds the index of the given movie title in the dataset and retrieves the similarity scores for that movie. The similarity scores are sorted in descending order, excluding the first entry (which corresponds to the movie itself).

The top N similar movies are selected based on the sorted similarity scores, and their titles are printed as recommendations.

C. Movie Review Engine:

It starts with the data preprocessing stage, in which several steps are performed to clean and standardize the dataset for accurate analysis.

This includes handling missing values and duplicates through functions like `dropna()` and `drop_duplicates()`. Tokenization is then applied to divide the text documents into individual words or tokens and stop word removal is performed to eliminate commonly occurring words like "and," "the," and "is" that do not carry significant meaning for sentiment analysis.

A visual summary of the most prominent words in the analyzed text data is then generated using the word cloud visualization technique. The size of each word in the word cloud represents its frequency or significance in the text, allowing for easy identification of major sentiments or topics.

Text vectorization using TFIDF Vectorizer transforms text documents into a matrix of numerical features using the term frequency (TF) and inverse document frequency (IDF) of words in the text.

Logistic regression is then used to classify text data based on sentiment by approximating the probabilities associated with each document belonging to the positive or negative sentiment category.

It transforms the result using the logistic function to generate a probability value between 0 and 1. The model is then trained, tested, and evaluated using the metrics of accuracy, precision, recall, and F1 score and confusion matrix.

New user reviews can be analyzed by loading the serialized model and predicting sentiment labels. The sentiment prediction output can be stored for further analysis and decision-making.

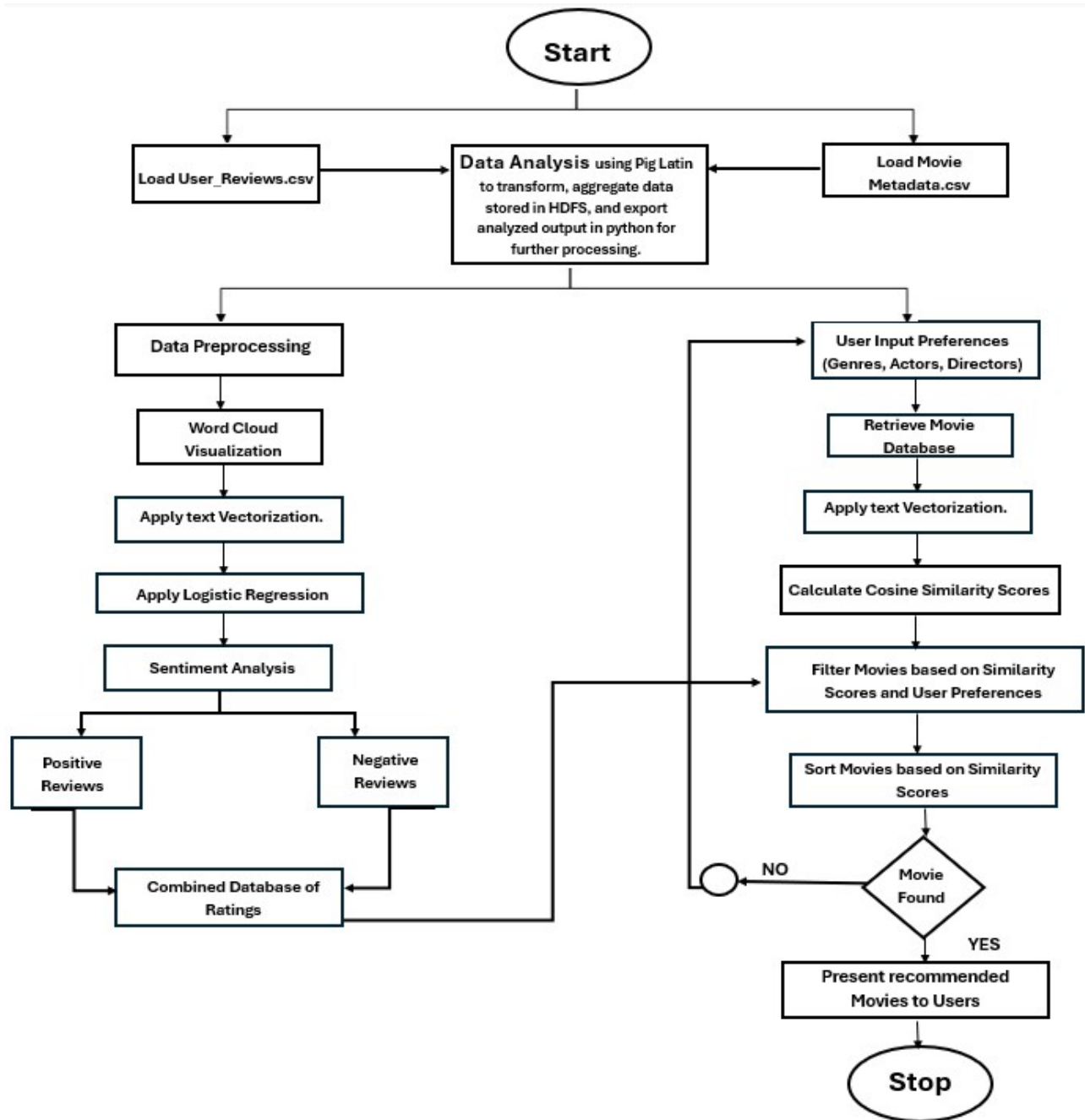


Figure 2.1 Movie Recommendation system and Review Engine

5 RESULTS AND DISCUSSION

5.1 Recommendation System

Firstly, the external libraries or JAR files are included to provide additional functionalities. Movie metadata and reviews data are analyzed using the Apache pig. It made it possible for the system to analyze data properly and manage massive amounts of user evaluations and movie metadata. The process began with loading the dataset in formats such as CSV, TSV, or JSON and

establishing the schema. The process began with loading the dataset in formats such as CSV, TSV, or JSON and then relevant columns were selected such as index, genres, original title etc. With the FILTER statement, we filtered the rows where the 'genres' column or the 'original_title' column containing the word "adventure." This allowed us to focus specifically on adventure movies. Next, we sorted the adventure movies based on their ratings in descending order. By using the LIMIT keyword,

we restricted the result to the top 10 rows, ensuring we only considered the highest-rated adventure

movies. The sorted movies, as shown in Fig 2.2 displayed as recommendations to the users.



Figure 2.2 Sorted movies based on average votes

The results of the analysis were stored separately for each genre. This dataset was saved as a CSV file in the specified HDFS location. Repeating the same process for other genres, including drama, action, thriller/mystery, horror, and crime. This allowed us to generate recommendations for each genre separately. The results of the analysis for each genre were stored in a separate dataset and saved as a CSV file in a specified HDFS location. Each section of the script followed a similar structure, performing similar operations, but for different genres. For example, we filtered, ordered, limited, and stored

the top 10 movies for adventures, action, thriller/mystery, drama, horror, and crime genres. Fig 2.3 of our paper shows the movies separated by different genres stored in HDFS. By organizing the recommendations based on specific genres and storing them separately, our recommendation system provides users with a more personalized and targeted movie suggestion experience. The process described ensures that the recommendations are relevant to the user's preferred genres and are based on high ratings.

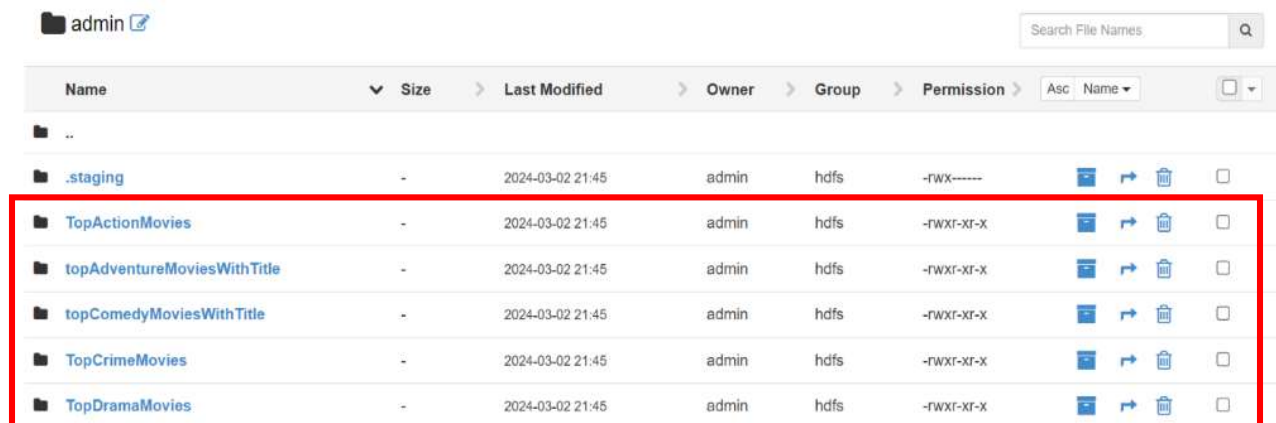


Figure 2.3 Movies separated by genres saved in HDFS

After analyzing data using pig Latin, we will then move this analyzed output forward for the data preprocessing in machine learning. Where on the basis of genres we will make recommendations of movies for the user. As the graph given below in Fig. 2.4 represents the distribution of movies across

different genres, with the x-axis indicating the genres and the y-axis representing the number of movies in each genre.

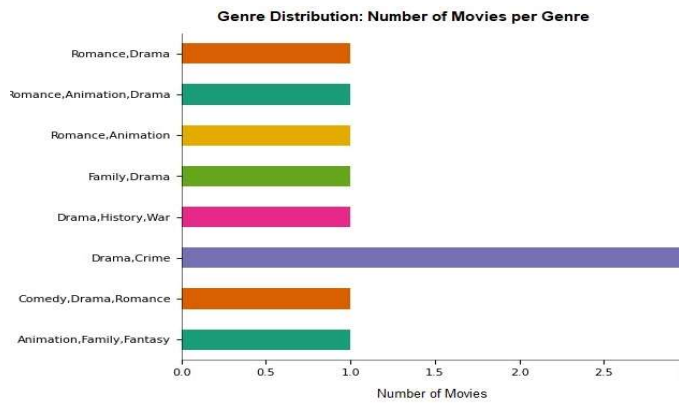


Figure 2.4 Number of movies per genre

Then Fig. 2.5 shows the graph which gives the visualization for the distribution of movies across different genres and how they are rated or voted by viewers.

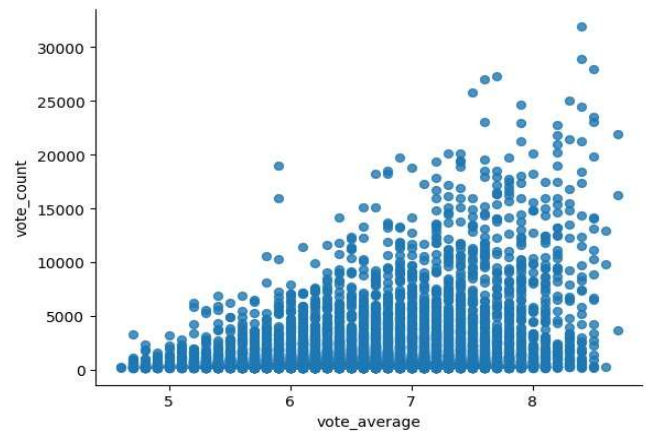


Figure 2.5 Distribution of movies across different genres

The output of the recommendation system is shown in Fig 2.6.

```

HYY!! WELCOME TO RECOMMENDATION SYSTEM
Enter the name of the Movie you want to watch: The Avengers

MOVIE SUGGESTED FOR YOU ARE :

1 . The Avengers
2 . Avengers: Age of Ultron
3 . Captain America: The Winter Soldier
4 . Captain America: Civil War
5 . Iron Man 2
6 . Thor: The Dark World
7 . X-Men
8 . The Incredible Hulk
9 . X-Men: Apocalypse
10 . Ant-Man
11 . Thor
  
```

Figure 2.6 Output of Movie Recommendation System

When a user searches for a specific movie, the system calculates the similarity between the searched movie and other movies in its database, considering factors like genre, director, actors, and plot keywords. It then generates a ranked list of recommended movies that are most similar to the searched movie, accompanied by relevant details such as title, genre, release year, and a synopsis. The system also takes into account the user's preferences and viewing history to further personalize the recommendations. The aim is to provide users with tailored suggestions that match their tastes and allow them to discover new movies aligned with their interests.

4.1 Review Engine

Review Engine is developed to analyze movie reviews data from the CSV file is loaded into HDFS USING PigStorage function with comma as the delimiter. The 'review' column was then tokenized using the FOREACH statement, resulting in a flattened relation with an additional column named 'words' containing individual tokens from the 'review' text. This step allowed us to break down the reviews into separate words for further analysis. Fig 2.7 shows the result of FOREACH statement used for tokenization.



Figure 2.7 FOREACH statement for tokenization

Reviews will be given sentiment ratings based on a sentiment dictionary that indicates whether a review is good, negative, or neutral. To assign sentiment ratings to reviews, we utilized a sentiment dictionary that indicates whether a review is positive, negative, or neutral. The sentiment dictionary was loaded from a file into a relation called 'dictionary'. Each word in the dictionary was associated with a sentiment rating, facilitating sentiment analysis or scoring. A left outer join was applied between the 'tokens' and 'dictionary' relations, matching the 'words' column from the 'tokens' relation with the 'word' column from the 'dictionary' relation. This join created a new relation called 'word_ratings', which combined the tokenized words with their corresponding sentiment ratings. Tuples in the 'word_ratings' relation were grouped based on the 'id' and 'review' columns, creating a new relation called 'group_words'. For each group in the 'group_words' relation, the average rating was calculated. The result was stored in the 'avg_ratings' relation, which included the 'group' column (containing the 'id' and 'review' values) and the calculated average rating as

'review_rating'. This step allowed us to determine the average sentiment rating for each review, providing an overall sentiment score.

To categorize reviews as positive or negative, we filtered the 'avg_ratings' relation. For positive reviews, we kept only the tuples where the 'review_rating' was greater than 0. Similarly, for negative reviews, we kept only the tuples where the 'review_rating' was less than 0. The filtered positive reviews were stored as 'positive_reviews', and the filtered negative reviews were stored as 'negative_reviews' in HDFS at the specified paths. The Review Engine enabled us to process and categorize movie reviews based on their sentiment ratings, as Fig. 2.8 and 2.9 showshow filtered positive and negative reviews respectively. Finally, reviews are stored as 'positive_reviews' and 'negative_reviews' in relation to the HDFS at the specified paths Fig. 2.10 shows the reviews stored in HDFS. After performing data analysis of movie reviews using pig latin export this analyzed output for further processing in Python for machine learning.

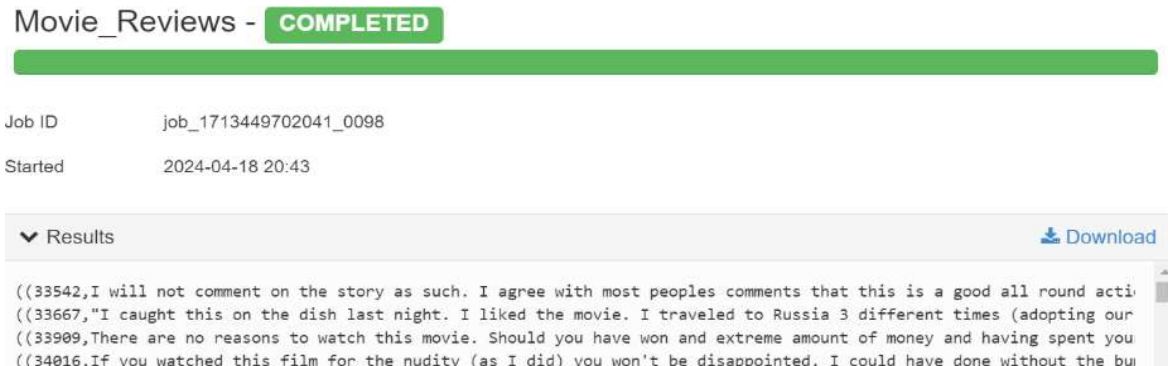


Figure 2.8 Filtered Positive Reviews

Movie_Reviews - COMPLETED

Job ID job_1713449702041_0101

Started 2024-04-18 20:45

▼ Results [Download](#)

```
((32823,Once again Elmer is faced with the dilemma of who to shoot. Bugs of Daffy. He's unsure of what season it is an
((33155,This movie was sweet. The main character lady was sensitive to 2 different men who wanted her. She seemed not
((33196,This film is the worst film I have ever seen. The story line is weak - I couldn't even follow it. The acting i
((33644,Just watched this film on TV and it was awesome. <br /><br />had just planned on watching it whilst doing some
((34069,(spoilers?)<br /><br />while the historical accuracy might be questionable... (and with the mass appeal of the
```

Figure 2.9 Filtered Negative Reviews

← → ↻ ⚠ Not secure 192.168.48.152:8080/#/main/views/FILES/1.0.0/Files ☆ ⬇ ⌵ 👤 ⋮

Gmail YouTube Maps All Bookmarks

Ambari Sandbox [login](#) [logout](#) Dashboard Services Hosts Alerts Admin [admin](#)

/ user / admin [+ New directory](#) [Browse...](#) [Select files to upload...](#)

[admin](#) Search File Names

Name	Size	Last Modified	Owner	Group	Permission	Asc	Owner
..							
.staging	-	2024-02-03 19:19	admin	hdfs	-rwxr-xr-x		
CosineSimilarity.jar	3.5 kB	2024-02-03 18:14	admin	hdfs	-rwxr-xr-x		
WordCount-5.1.2.jar	1.6 MB	2024-02-03 18:13	admin	hdfs	-rwxr-xr-x		
commons-lang-2.6.jar	277.6 kB	2024-02-03 18:12	admin	hdfs	-rwxr-xr-x		
concat-0.1.14.jar	2.9 kB	2024-02-03 18:14	admin	hdfs	-rwxr-xr-x		
guava-13.0.1.jar	1.8 MB	2024-02-03 18:13	admin	hdfs	-rwxr-xr-x		
negative_reviews	-	2024-02-03 19:18	admin	hdfs	-rwxr-xr-x		
positive_reviews	-	2024-02-03 19:18	admin	hdfs	-rwxr-xr-x		

Figure 2.10 Reviews stored in HDFS

The bar graph showing in Fig 2.11 two bars representing different sentiments. The labels on the x-axis indicate "Positive" and "Negative," indicating the type of sentiment being measured. The y-axis represents the percentage of each sentiment. The graph suggests that there is a higher percentage of negative sentiment compared to positive sentiment. Where the result of sentiment analysis accurately predicts the sentiment expressed in user reviews of movies. The model successfully identifies and classifies the sentiment as positive, negative, or neutral.

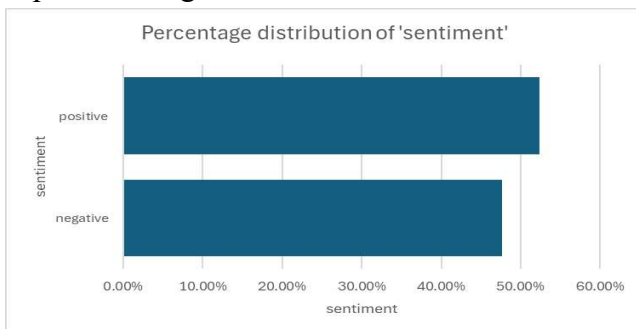


Figure 2.11 Percentage distribution of sentiments

Once the user submits a movie review, the sentiment analysis algorithm processes the input using the trained model. It maintains a dictionary of commonly occurring words of English language and give scores of all the words.

After this it predicts the overall sentiment as either negative or positive. Fig. 2.12 and 2.13 is the output shown below, it takes the review from the user and after processing all the words it predicts the sentiment as positive and negative.

Welcome to the Movie Review Sentiment Predictor!
Please enter your review about the movie:
> This movie is really good, performances are great.
The sentiment of the review is: positive

Figure 2.12 Positive Review Prediction

Welcome to the Movie Review Sentiment Predictor!
Please enter your review about the movie:
> There is nothing good in this Movie.
The sentiment of the review is: negative

Figure 2.13 Negative Review Prediction

5.2 Model testing/evaluation criteria:

5.2.1 Confusion Matrix

Confusion matrix is used to access the accuracy and performance of the sentiment analysis algorithm by evaluating the distribution of prediction across different classes. Fig. 2.14 below identifies the number of correct and incorrect prediction effectively.

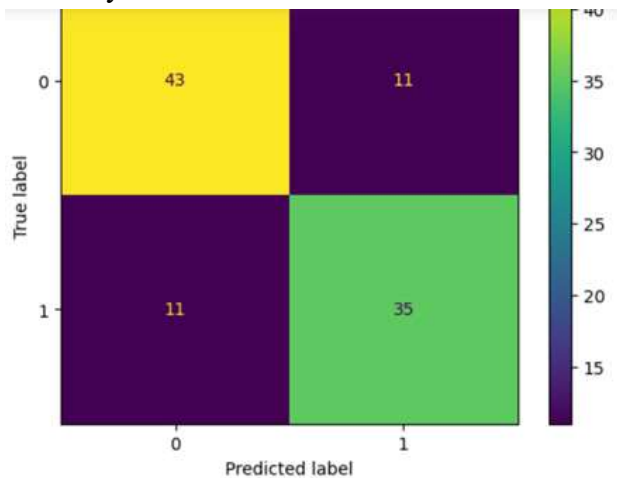


Figure 2.14 Confusion Matrix

5.2.2 Accuracy

By comparing the predicted sentiment labels with the ground truth labels, a confusion matrix will be used to assess the accuracy of the sentiment analysis component and reveal information about the algorithm's performance. The formula for calculating accuracy is in equation 1.9.

$$Accuracy = \frac{(TP+TN)}{(TP+TN+FP+FN)} \quad 1.9$$

By putting values in the above equation, the value of accuracy is yielded as 0.54 or 54% which shows a moderate level of accuracy in the system.

5.2.3 Precision

It is used to measure the percentage of accurately predicted positive cases among all the instances that are projected to be positive. It is calculated by using equation 1.10 for the above confusion matrix and resulted in 0.554 or 55% for the sentiment analysis review system.

$$Precision = \frac{TP}{(TP+)} \quad 1.10$$

5.3 User Interface:

The recommendation server has an input field where the users can enter the movies they want to

watch and by clicking the enter button, they can get the recommendations. Fig. 2.15 shows the recommendation page along with the searched movie's metadata, fig. 2.16 shows the cast and crew and user reviews while recommended movies are displayed in Fig. 2.17. Other than this, the review prediction as either positive or negative is shown in Fig. 2.18 and 2.19.

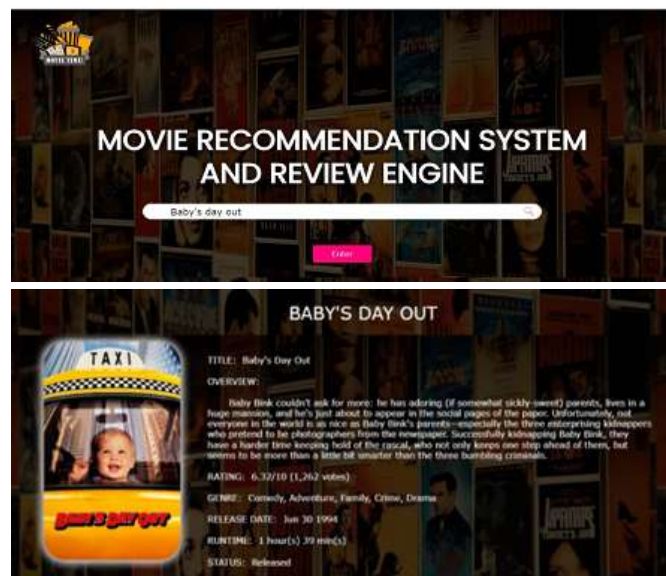


Figure 2.15 Movie's metadata



Figure 2.16 Cast and crew and user reviews

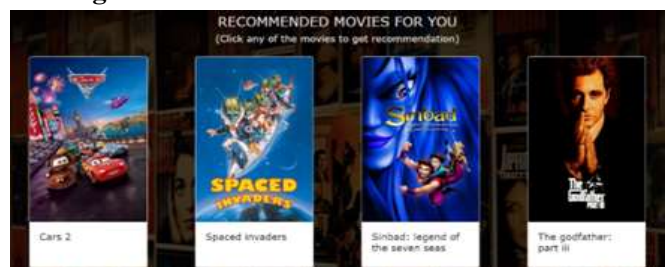


Figure 2.17 Recommended Movies

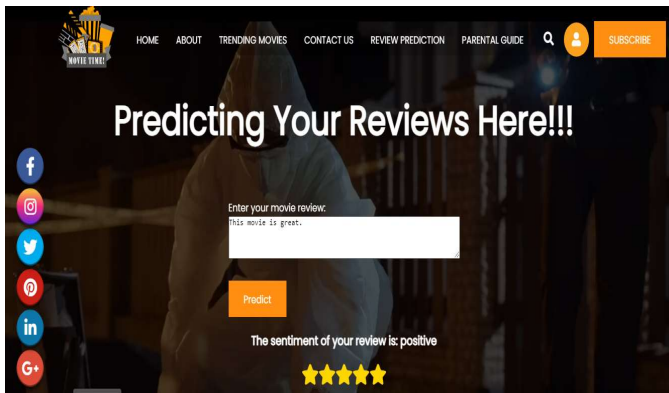


Figure 2.18 Positive Review Prediction

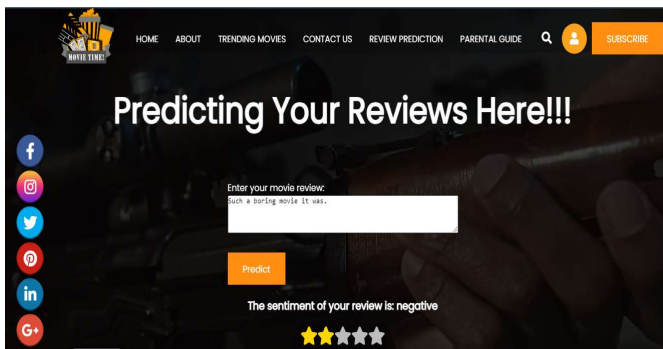


Figure 2.19 Negative Review Prediction

6. Conclusion:

To improve the user experience, sentiment analysis and content-based movie recommendation algorithms are combined in the "Big Data-based IMDb Movie Review Engine". Based on user preferences and perceptive attitude analysis, the engine uses big data to offer tailored movie recommendations.

The movie recommendation and sentiment analysis domains benefit from the system's contribution to the process of finding movies. Expanding the dataset, improving algorithms, and adding user evaluations and social media data are possible areas of future study to improve performance even further. The engine, taken as a whole, improves user pleasure and the cinematic experience.

ACKNOWLEDGMENT

We would like to acknowledge our research advisor, Dr. Farhan Qamar, department of Computer Engineering, University of Engineering and Technology Taxila for his guidance and support throughout the research process. We are grateful to him for assisting us with the data collection and analysis.

REFERENCES

- [1] Skrzypczyński, Mikołaj, and Piotr Muryjas. "Analysis of data processing efficiency with use of Apache Hive and Apache Pig in Hadoop environment." *Journal of Computer Sciences Institute* 30 (2024): 1-8.
- [2] Singh, Ranjeet Kumar. "Developing a big data analytics platform using Apache Hadoop Ecosystem for delivering big data services in libraries." *Digital Library Perspectives* (2024).
- [3] Verma, Jai Prakash, Bankim Patel, and Atul Patel. "Big data analysis: recommendation system with Hadoop framework." *2015 IEEE International Conference on Computational Intelligence & Communication Technology*. IEEE, 2015.
- [4] Hongyan Liu, Jun He, Tingting Wang, Wenting Song, Xiaoyang Du, "Combining user preferences and user opinions for accurate recommendation", Elsevier Journal- Electronic Commerce Research and Applications 12 (2013) 14–23.
- [5] Sarker, Kamal Uddin, et al. "A Ranking Learning Model by K-Means Clustering Technique for Web Scraped Movie Data." *Computers* 11.11 (2022): 158.
- [6] B. Pang, L. Lee, and S. Vaithyanathan, "Tumbs up?: sentiment classification using machine learning techniques," in *Proceedings of the ACL-02 Conference on Empirical Methods in Natural Language Processing—Volume 10 (EMNLP '02)*, pp. 79– 86, Association for Computational Linguistics, Stroudsburg, Pa, USA, July 2002.
- [7] P. D. Turney, "Tumbs up or thumbs down?" in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, pp. 417–424, Philadelphia, Pennsylvania, July 2002.
- [8] Rajarajeswari, S., et al. "Movie recommendation system." *Emerging Research in Computing, Information, Communication and Applications: ERCICA 2018, Volume 1*. Springer Singapore, 2019.
- [9] Peng, Yuanzhe. "A survey on modern recommendation system based on big data." *arXiv preprint arXiv:2206.02631* (2022).

- [10] Wei, Chutian, et al. "Fully content-based IMDb movie recommendation engine with Pearson similarity." *International Conference on Green Communication, Network, and Internet of Things (GCNIoT 2021)*. Vol. 12085. SPIE, 2021.
- [11] Budiprasetyo, Gunawan, et al. "Big data system of research data in the informatics department based on software enhancement." *International Journal CISIMA* (14) (2021): 347-362.
- [12] Wang, Yibo, Mingming Wang, and Wei Xu. "A sentiment-enhanced hybrid recommender system for movie recommendation: a big data analytics framework." *Wireless Communications and Mobile Computing* 2018 (2018).
- [13] N. Entezari, E. E. Papalexakis, H. Wang, S. Rao, and S. K. Prasad, "Tensor-based complementary product recommendation," in 2021 IEEE International Conference on Big Data (Big Data). IEEE, 2021, pp. 409–415.
- [14] S. Panigrahi, R. K. Lenka, and A. Stitipragyan, "A Hybrid Distributed Collaborative Filtering Recommender Engine Using Apache Spark," in *Proceedings of the 7th International Conference on Ambient Systems, Networks and Technologies, ANT 2016 and the 6th International Conference on Sustainable Energy Information Technology, SEIT 2016*, pp. 1000–1006, Spain, May 2016.
- [15] Singh, Ramni Harbir, et al. "Movie recommendation system using cosine similarity and KNN." *International Journal of Engineering and Advanced Technology* 9.5 (2020): 556-559.
- [16] Lütke, Marvin. *Calculate Similarity - the Most Relevant Metrics in a Nutshell*. 2021.
- [17] Dahir, Ubaid Mohamed, and Faisal Kevin Alkindy. "Utilizing Machine Learning for Sentiment Analysis of IMDB Movie Review Data." vol 71 (2023): 18-26.
- [18] Wang, Jie, and Zihao Li. "Research on face recognition based on CNN." *IOP Conference Series: Earth and Environmental Science*. Vol. 170. No. 3. IOP Publishing, 2018.
- [19] Tommandru, Suresh, and Domnic Sandanam. "An Automated Framework for Patient Identification and Verification Using Deep Learning." *Revue d'Intelligence Artificielle* 34.6 (2020).
- [20] Reddy Boyapally, Shashank. "Facial Recognition and Attendance System Using Dlib and Face_Recognition Libraries." Available at SSRN 3804334 (2021).
- [21] C. Basu, H. Hirsh, and W. Cohen, "Recommendation as classification: Using social and content-based information in recommendation," in *Proceedings of the 1998 15th National Conference on Artificial Intelligence, AAAI*, pp. 714–720, July 1998.
- [22] C. Ono, M. Kurokawa, Y. Motomura, and H. Asoh, "A contextaware movie preference model using a bayesian network for recommendation and promotion," in *Proceedings of 11th International Conference on User Modeling*, pp. 247–257, 2007
- [23] Abubakar, Haisal Dauda, Mahmood Umar, and Muhammad Abdullahi Bakale. "Sentiment classification: Review of text vectorization methods: Bag of words, Tf-Idf, Word2vec and Doc2vec." *SLU*
- [24] *Journal of Science and Technology* 4.1 & 2 (2022): 27-33. Peng, Yuanzhe. "A survey on modern recommendation system based on big data." *arXiv preprint arXiv:2206.02631* (2022).
- [25] *Movie Recommendation System* S. Rajarajeswari, Sharat Naik, Shagun Srikant, M. K. Sai Prakash and Prarthana Uday
- [26] Y. Cheng and X. Bu, "Research on key technologies of personalized education resource recommendation system based on big data environment," in *Journal of Physics: Conference Series*, vol. 1437, no. 1. IOP Publishing, 2020, p. 012024.
- [27] Gavua, Ebenezer Komla, Seth Okyere-Dankwa, and Gignarta Kasaye Soka. "Big Data Sentiment Analysis based on PLSA and its Application."
- [28] Gupta, Urvashi, and Rohit Sharma. "Apache Hadoop framework for big data analytics using AI." *Artificial Intelligence and Blockchain in Industry 4.0*. CRC Press, 2024. 130-14